ONE MODEL. SIMULATE. FIT. OPTIMIZE.

# Hands-On with Rumoca

**James Goppert · Micah Condie**

Purdue University · School of Aeronautics and Astronautics

Georgia Tech · 10:00–13:00

Any modeling domain welcome · no aerospace background needed

**MODEL**
Declare the physics

**FIT**
Learn from data

**OPTIMIZE**
Plan the motion



CREATE CORE CODESPACE

## Scan to create the core Codespace · repository link above

**Repository ↗** **Workshop guide ↗** **Create Codespace ↗**

# Run the core. Explore your own question.

YOUR QUESTION. YOUR EXPERIMENT.

## Make something you care about.

Experiment in a model or simulator.

Core: edit a model, fit data, optimize a trajectory

One change in an existing model or simulator is enough

THE PRIZE

**FREE**

**CogniPilot stocking cap**

---

Short demos
Audience hand vote

**Enter by 12:40 · up to 5 demos · CHALLENGE.md · hosting optional**

**Challenge rules ↗** **Example prompts ↗**

# Why Rumoca? One model, many questions.

## A unified, declarative foundation for cyberphysical systems.

### Lie group theory

Represent rotations and motion with their geometric structure.

### Reachable sets

Bound possible behavior of cyberphysical systems under uncertainty.

### Formal software verification

Connect model equations to explicit proof obligations.

### System identification + digital twins

Fit physical parameters; compare simulation with experiments.

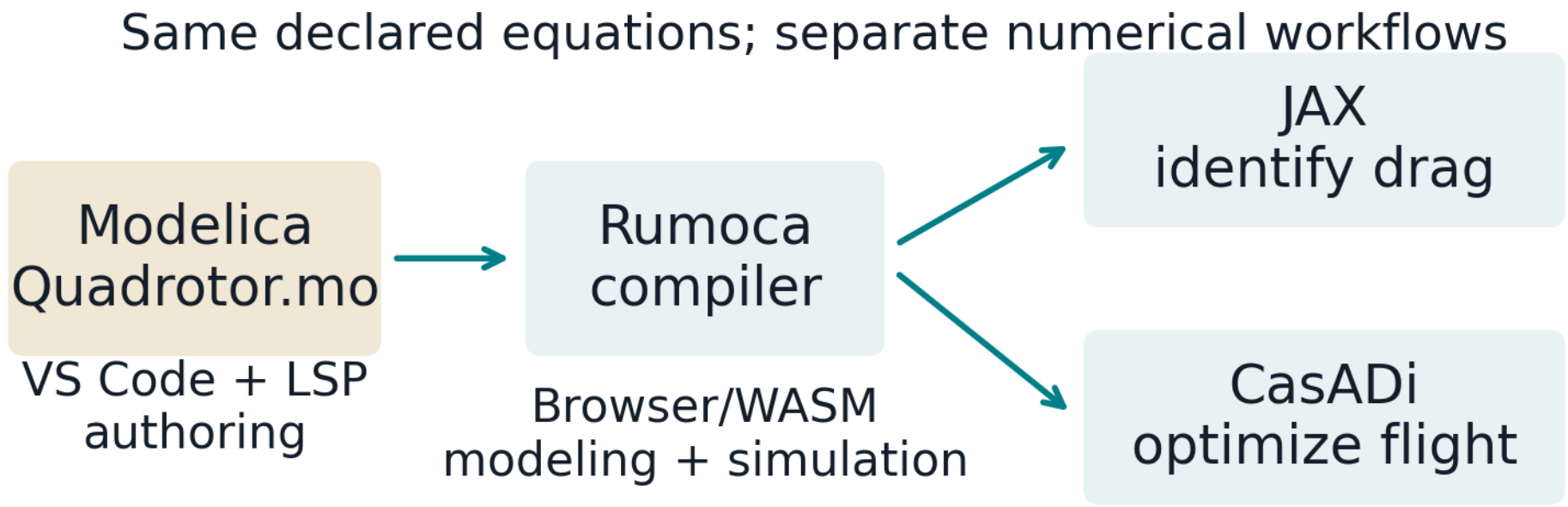
**Cerebri RDD2: Modelica → eFMI C → Zephyr RTOS → real drone**

**Goal: mathematical guarantees for specified properties under explicit assumptions.**

Rumoca compiler ↗ | Model library ↗ | Cerebri RDD2 ↗

# Declare once. Ask different questions.

## Quadrotor.mo → JAX + CasADi · the same declared equations

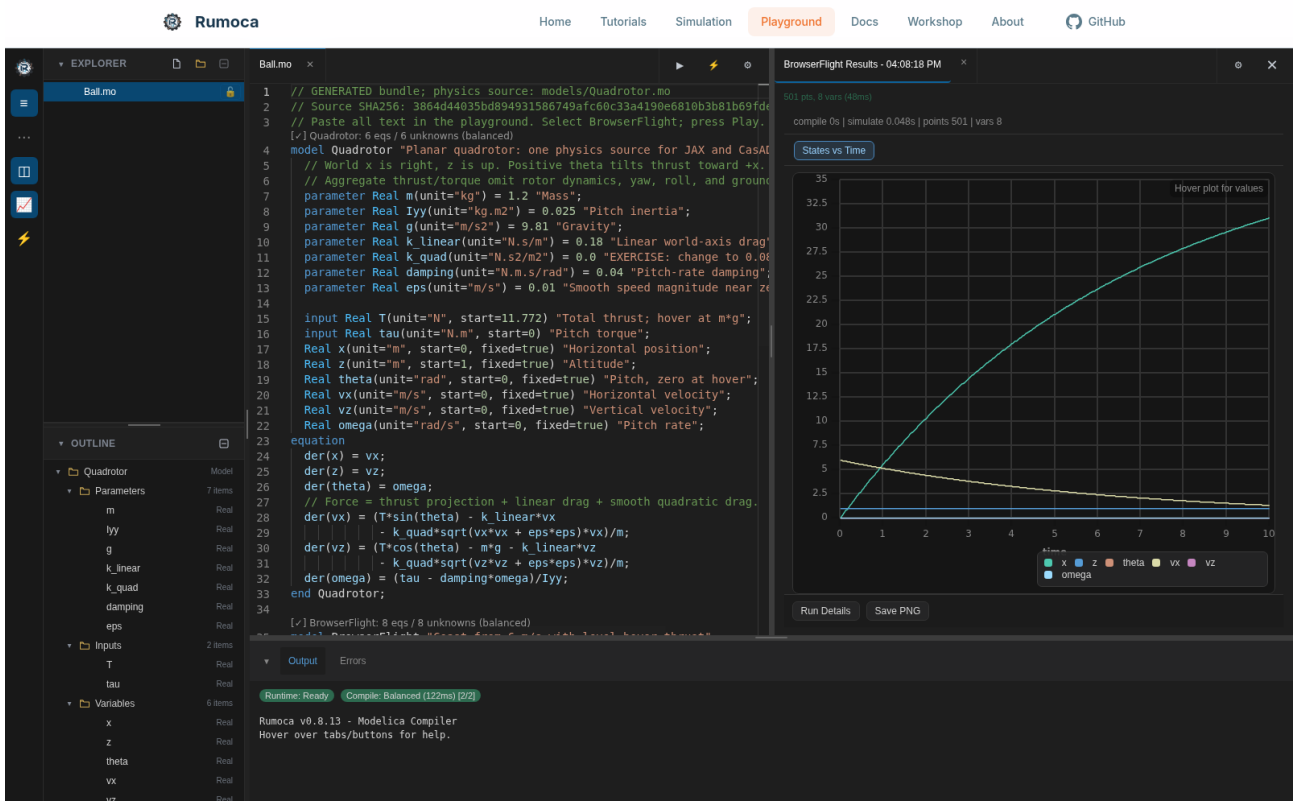



## Modelica declares physics · Rumoca compiles · backends compute.

# Everyone gets a browser plot

**TRY THIS**

1. Open the playground
2. Paste the complete browser bundle
3. Gear → BrowserFlight → Play




**You should see BrowserFlight Results and a plot.**

**Open playground ↗**

**Browser model ↗**

# Choose a full-workshop route

**01** Browser: modeling and predictions; no account

**02** Codespaces: Python + VS Code; login and allowance

**03** Local uv: Python + VS Code; pinned CPU packages

**04** Pair or screenshare: keep working if setup fails

**Switch route after ten minutes of setup trouble.**

Local VS Code setup ↗ | Create Codespace ↗ | Recovery help ↗

# Configure the experiment, then run it

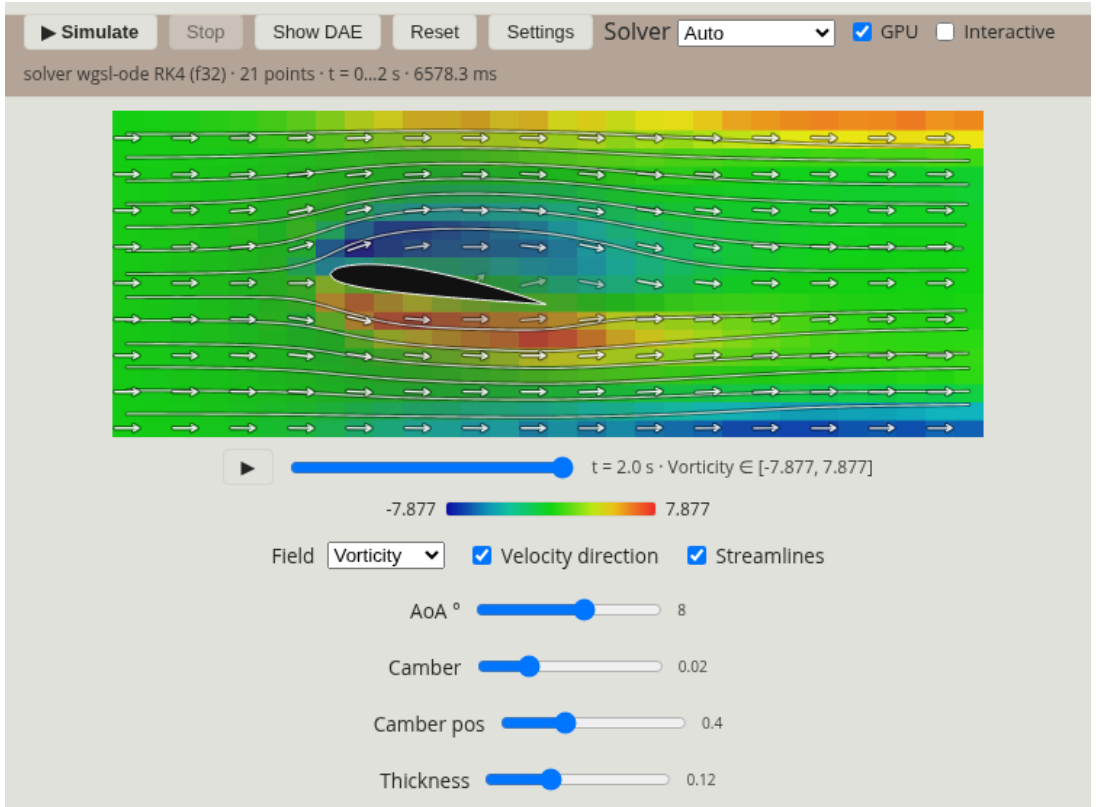
1. Open BrowserFlight.mo → click the lightning bolt
2. Save scenario TOML → model, source roots, solver, end time
3. Run the scenario → inspect vx and x

**Modelica owns physics · TOML owns the run configuration.**

# The user guide is a live modeling lab

**TRY THIS**

1. Edit StopTime: 30 → 2 s
2. Simulate; select the final frame
3. Switch vorticity / pressure




## User-guide link and recovery: WORKSHOP.md · no install needed

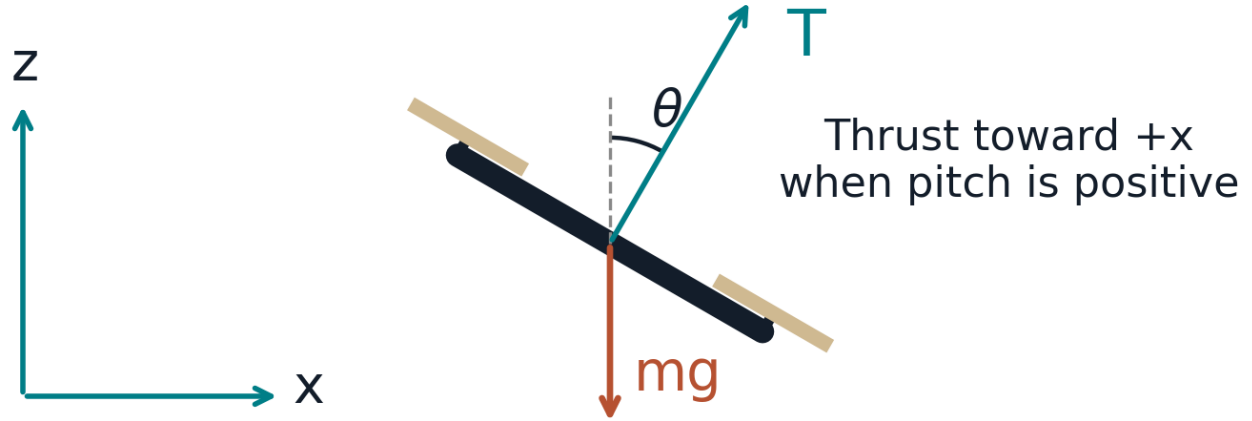
**Open NACA example ↗** **Explore user guide ↗**

# A planar quadrotor we can inspect

READ THE MODEL

1. World x right; z up
2. Positive pitch tilts thrust toward +x
3. Hover: T = mg = 11.772 N

Planar motion; ideal aggregate thrust and torque




**Find velocities, pitch rate, thrust and torque in the model.**

Open vehicle model ↗

Workshop guide ↗

# Your first engineering change: drag

**Change k_quad = 0.0 to 0.08, save, then simulate.**

$$D(v) = -k_{\text{linear}} v - k_{\text{quad}} \sqrt{v^2 + \varepsilon^2}\, v$$

Linear: N s/m  Quadratic: N s²/m²  ε = 0.01 m/s

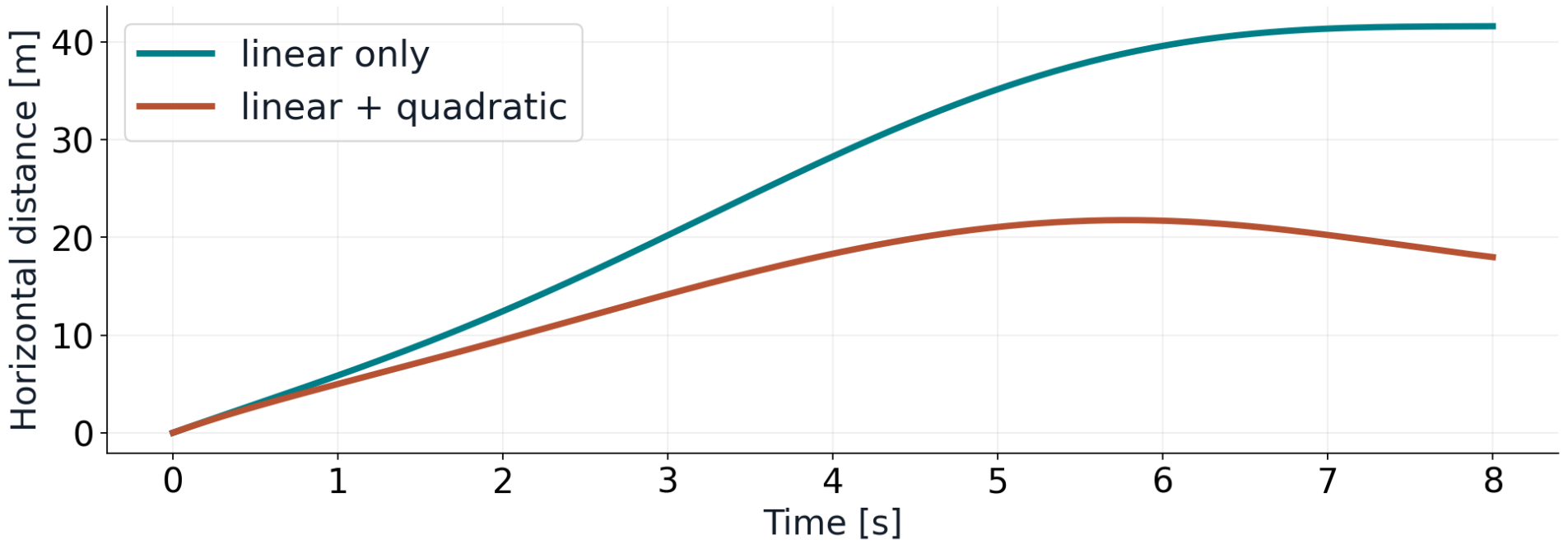
Nonnegative coefficients ⇒ drag opposes motion

```
$ uv run --frozen python examples/simulate.py
```

**What changes at high speed? What about negative velocity?**

# More drag, less distance

## Reference at 8 s: 41.60 m → 17.97 m




## Compare your current default with the two reference cases.

# Back at

# 11:10

**10 MINUTES**

Stretch. Ask a question. Keep your editor open.

**KEEP YOUR PLACE**

Keep your editor and terminal open.

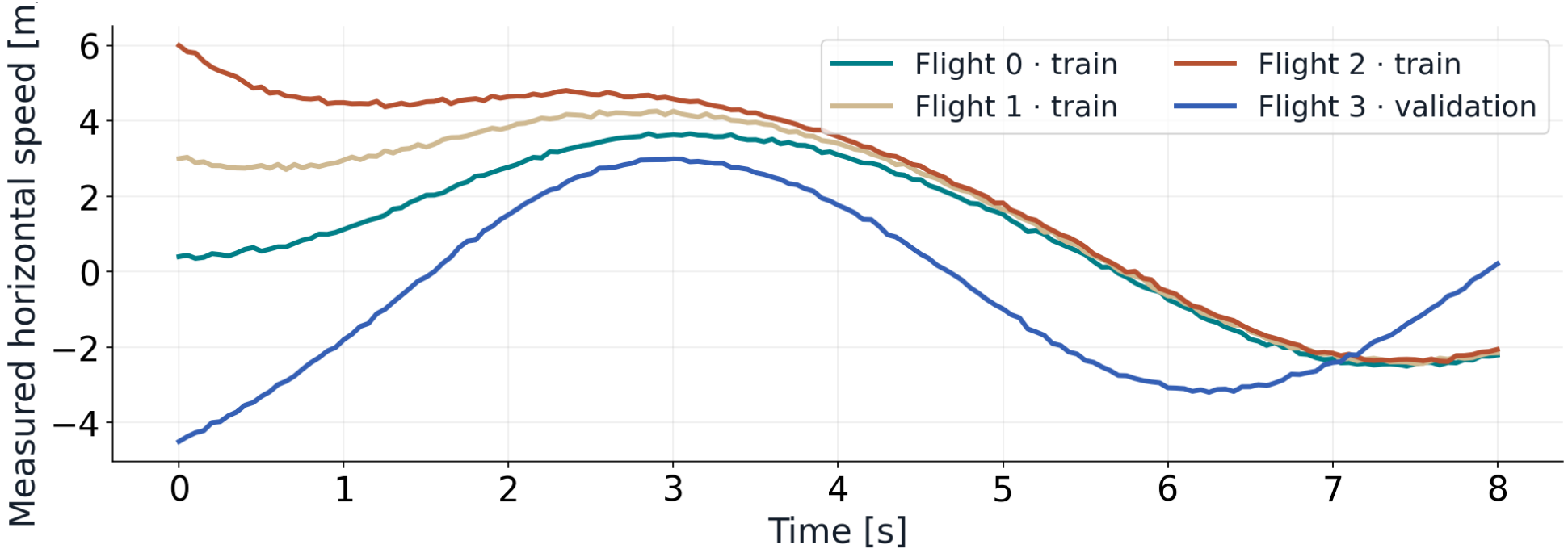
Your simulation plot is the recovery point.

Next: learn drag from noisy data.

**Worksheet and commands: WORKSHOP.md · scan the opening QR.**

# Known experiment. Unknown coefficients.

## Three training flights; one held-out flight. 20 Hz, 8 s each.




## Speed variation helps separate linear and quadratic drag.

# Differentiate the prediction error

## Positive parameters; noise-scaled loss; bounded optimization

Generated RHS Modelica → JAX → RK4 rollout positive drag → Data residuals scaled by noise → JAX gradients L-BFGS-B

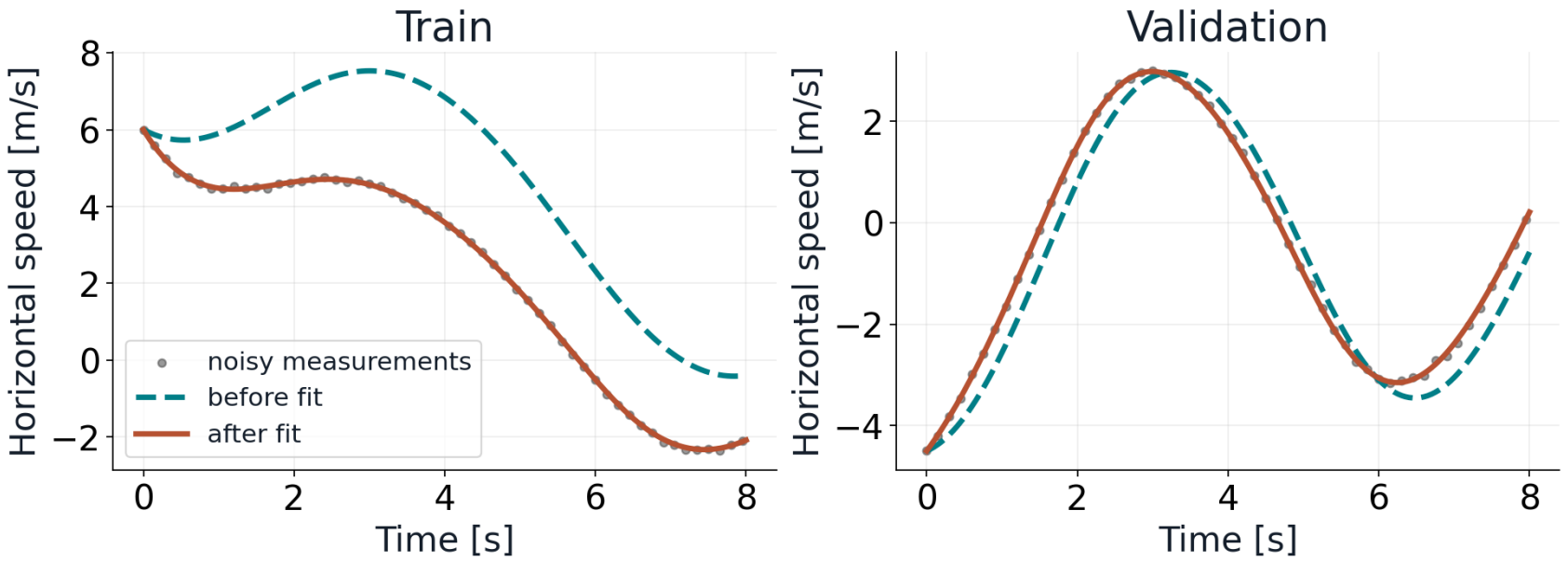
$$L = \mathrm{mean}\left[\left(\frac{\hat{y} - y}{\sigma}\right)^2\right]$$

$$k_l / k_{l,\mathrm{ref}} = e^a, \quad k_q / k_{q,\mathrm{ref}} = e^b$$

**JAX provides gradients. SciPy updates the coefficients.**

# Run the fit. Test the held-out flight.

## Open fit_drag.png and fit_drag.json; compare before and after.




```
$ uv run --frozen python examples/fit_drag_jax.py
```

## Two coefficients · lower loss · improved held-out prediction

# Check your result. Keep building.

**01** Open your before-and-after prediction plot

**02** Compare the held-out flight and fitted values

**03** Stretch: rerun from another starting guess

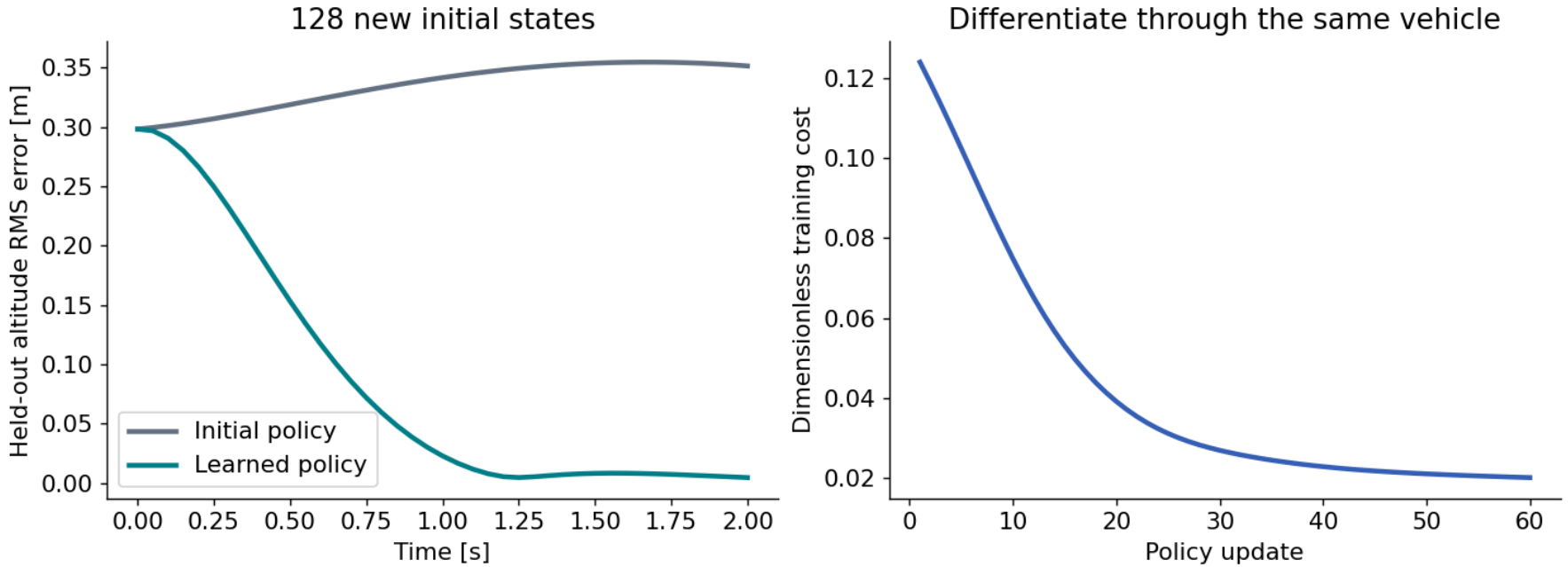
**04** App builders: keep your prototype moving

**Fit complete? Continue your experiment or explore the optional checks.**

Workshop guide ↗

# Make a target. Learn a policy.

## target.toml → generated Torch → differentiable policy learning




```
$ uv run --frozen --group torch python scripts/learn_hover_torch.py
```

**Held-out cost ≈ 0.116 → 0.021 · optional instructions: BACKENDS.md**

Backend walkthrough ↗ | Torch target.toml ↗ | Lean Jinja target ↗

# Back at

# 12:10

**10 MINUTES**

Stretch. Ask a question. Keep your editor open.

**KEEP YOUR PLACE**

Keep your fitted coefficients.

Next: plan a flight using your fit.

Your fit plot is the recovery point.

**Same Modelica source; now compile to CasADi.**

# Move between hovers, respect limits

THE IDEA

1. (0, 1) m → (4, 2) m in 6 s
2. Track target; limit effort and control changes
3. Bound thrust, torque, pitch and altitude




T: 0–20 N  τ: ±0.35 N m

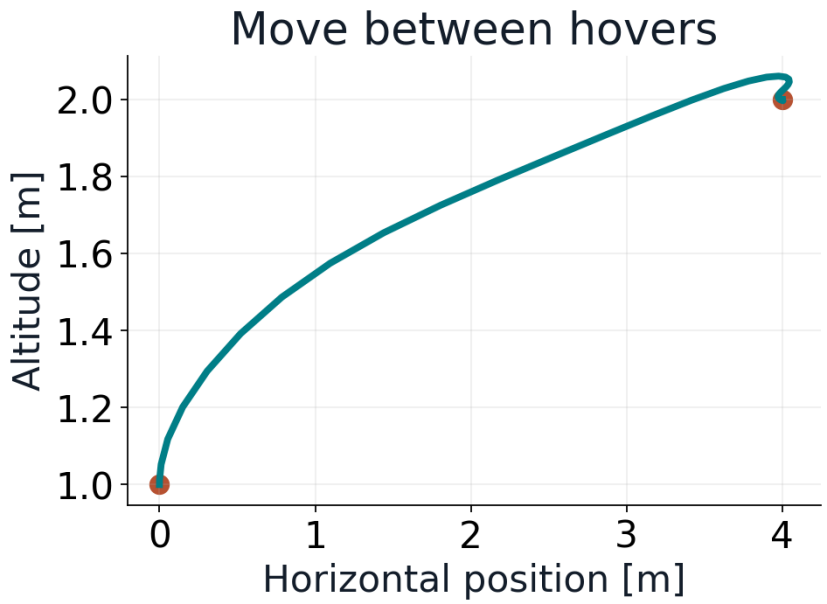
z: 0.25–4 m  |θ| ≤ 0.55 rad

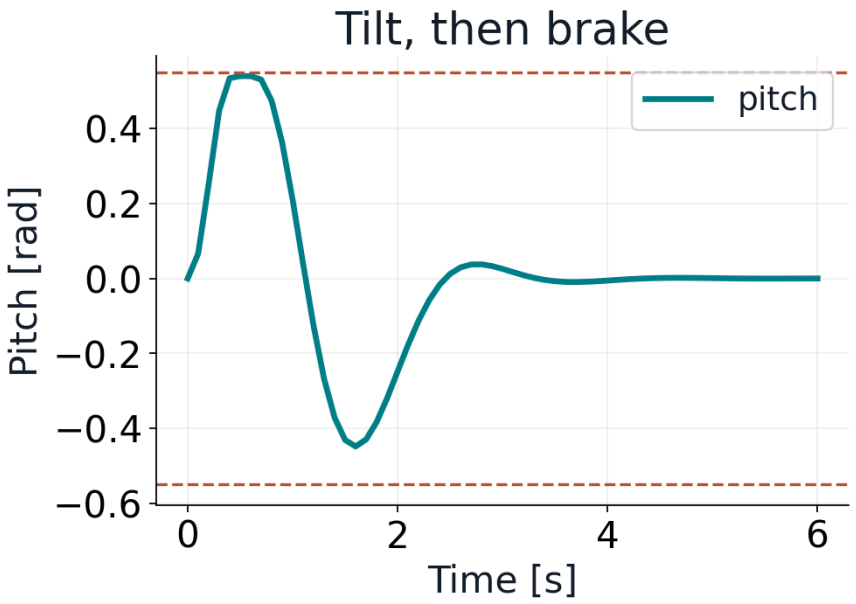
60 shooting intervals · node pitch margin · independent reintegration

**Identified drag is an input to this optimization.**

# Solve. Inspect. Explain the braking.

## Open optimization.png and optimization.json.




```
$ uv run --frozen python examples/optimize_casadi.py
```

**Solve_Succeeded · limits respected · arrive near rest**

**Workshop guide ↗**

# Next up: pylon racing at PURT

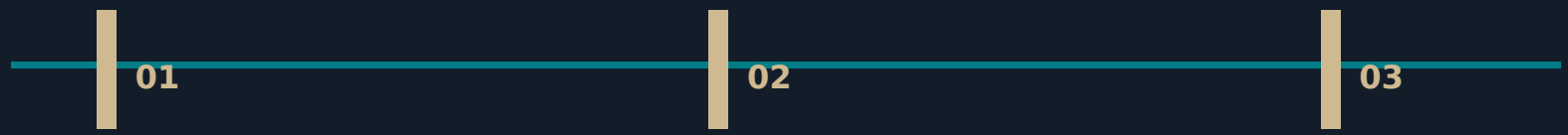
FROM DIGITAL TWIN TO THE RACE COURSE

# Build. Fly. Race.

## The next competition will leverage Rumoca

Design, simulate, then flight-test your autonomy

Digital twin → real aircraft at PURT




ILLUSTRATIVE COURSE · EVENT DETAILS ON THE SITE

FOLLOW THE
NEXT COMPETITION

**pylon-racing.geddes.rcac.purdue.edu**

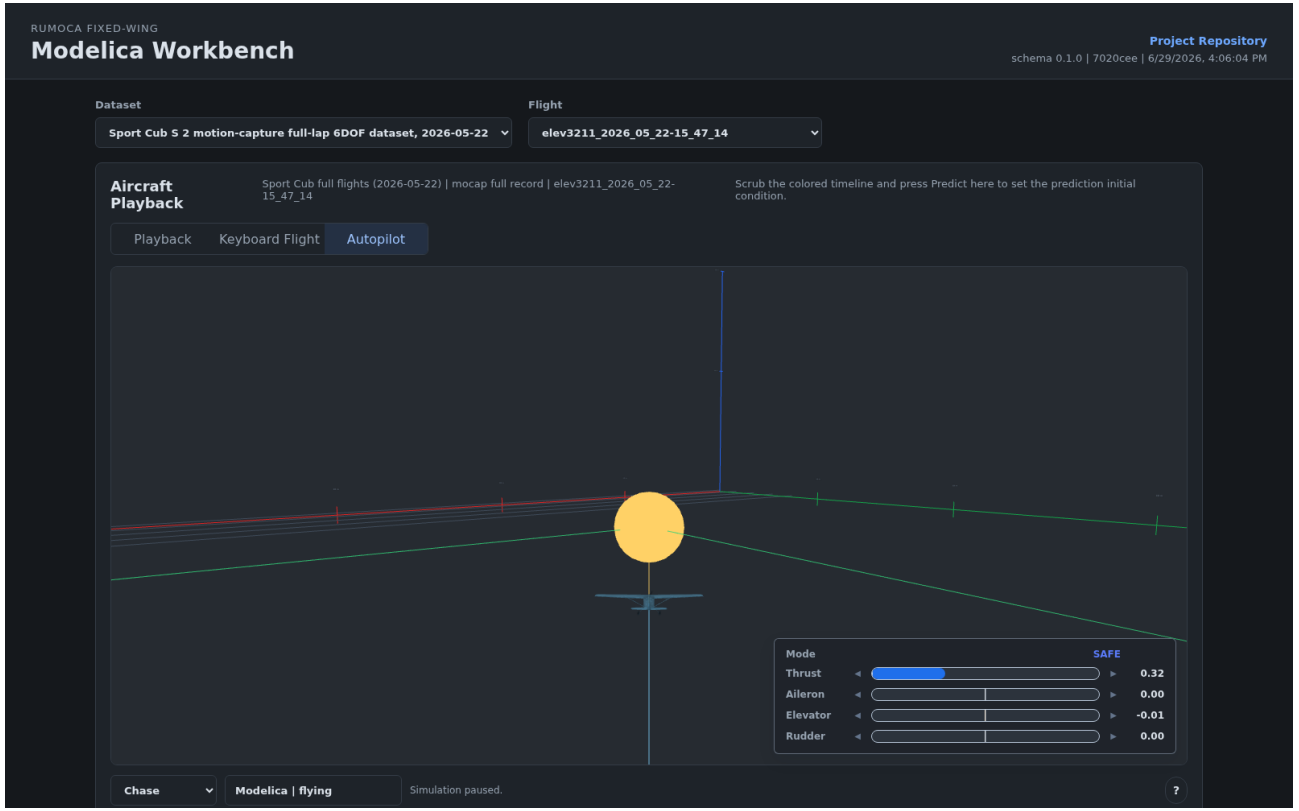
PURT pylon racing ↗

Challenge rules ↗

# Explore the fixed-wing workbench

**TRY THIS**

1. Modelica flight simulation + autopilot
2. Try 50 Hz → 20 Hz, restart, then observe
3. Also: slam_web (in development), mocap_sysid




## Live links + short experiments: BROWSER_APPS.md

Fixed-wing demo ↗ | Fixed-wing code ↗ | Mocap demo ↗ | Mocap code ↗ | SLAM (in dev) ↗ | SLAM code ↗

# Show your experiment. Vote for the cap.

| 5 | 45 s | 1 |
| --- | --- | --- |
| MAX. PRESENTERS | EACH DEMO | VOTE PER PERSON |

## Show your change + one checked result

Name your tool. Share elapsed work time.

**Audience favorite wins a free CogniPilot stocking cap**

**Random draw if more than 5 ready entries → audience hand vote**

Challenge rules ↗ Optional gallery ↗

# Bring your model.
# Join the work.

1. Bring a model or a question from your own domain
2. Add a validation case, model library or compiler fix
3. Start a conversation: problem + model + one check

James Goppert · Micah Condie

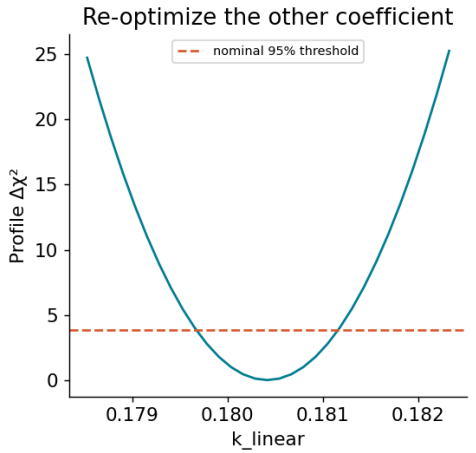
**Scan to join the work · COMMUNITY.md · questions welcome**

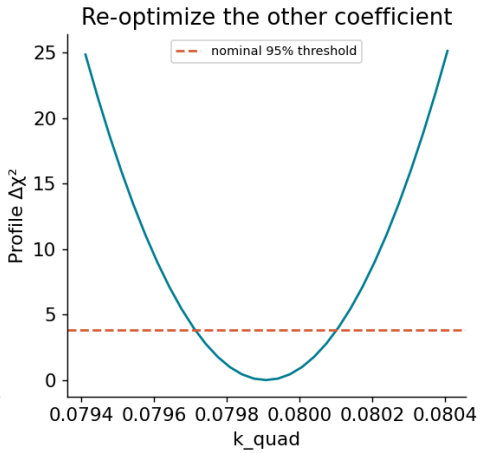
**Join the community ↗** | **Rumoca compiler ↗** | **Model library ↗**

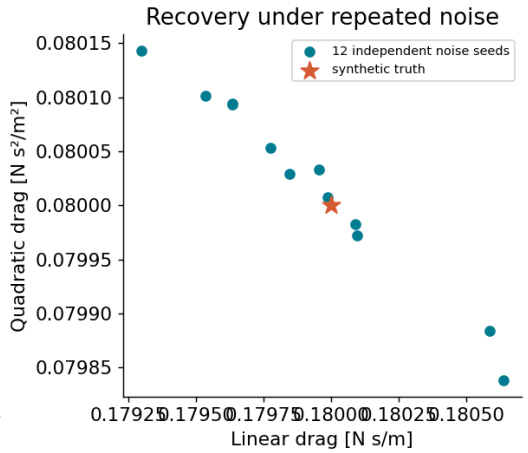
# Exact-real proofs and numerical evidence

THE IDEA

1. Lean: equation equality and invariants
2. Audit: gradients, profiles, integration
3. Papers: model and method choices




**None of these certify arbitrary edits or real flight.**

Lean proof source ↗ | Math evidence ↗

# Primary papers behind the approach

1. Faessler et al., RA-L 2018: rotor-drag physics and excitation
2. Andersson et al., 2019: CasADi AD and multiple shooting
3. Raue 2009; Wieland 2021: practical identifiability and profiles

**World-axis teaching drag differs from body-axis rotor aerodynamics.**

# FastDyn: fly the firmware against declared physics

**01** Rumoca compiles the Modelica plant into an FMI 3 FMU.

**02** FastDyn couples sensors and actuators to ArduPilot firmware in QEMU.

**03** Instruction-counted virtual time coordinates firmware and plant steps.

**04** New with options → select FastDyn + Rumoca. Run three load trials.

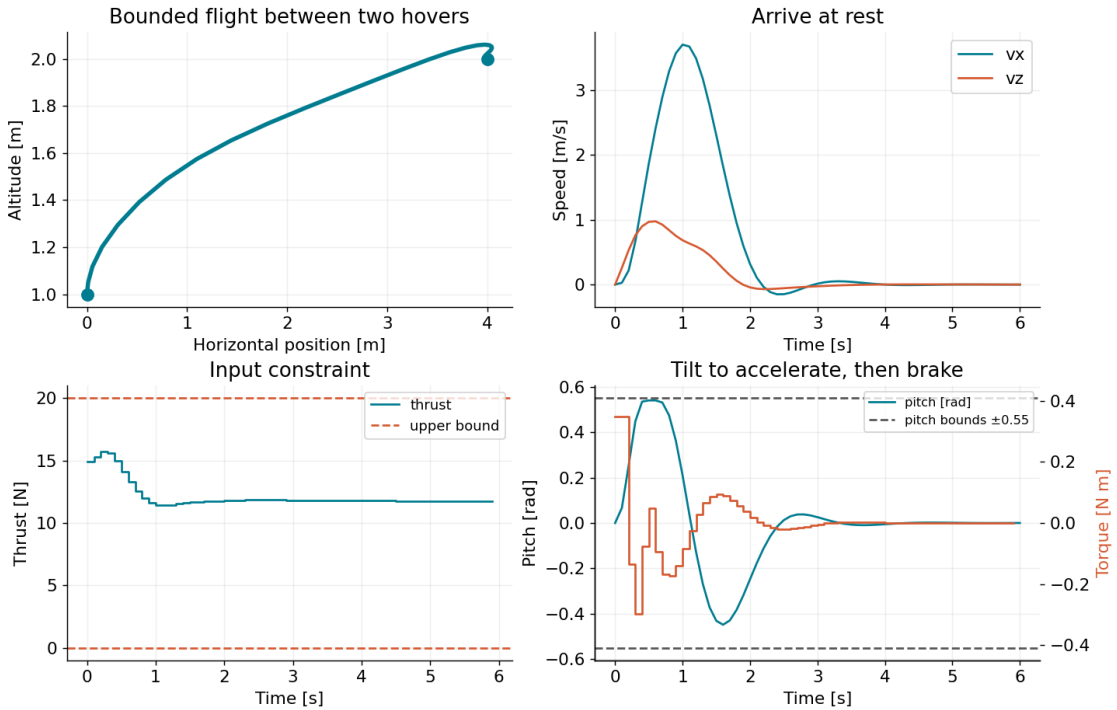
```
$ uv run --frozen python scripts/fastdyn_demo.py
```

**Choose the FastDyn container · open results/live/fastdyn/trajectories.png**

FastDyn demo guide ↗ | BoardRunner paper ↗ | FastDyn source ↗

# Numerical constraints need validation

## Reintegrate the inputs independently; inspect errors by physical unit.




## Small defects, pitch margin and fine rollout are separate checks.

**Numerical evidence ↗**